Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 5

Due date: Monday 11 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Parallel Space Solution of a nonlinear PDE using MPI [in total 60 points]

1.1. Initialize/finalize MPI and welcome message [5 Points]

First we are asked to complete the initialization and finalization of MPI in the main function, and to print a welcome message with the simulation parameters in the file `main.cpp`.
In listings 1 we can see the initialization of MPI. We initialize MPI with `MPI_Init`, and then we query the number of processes and the rank of each process with `MPI_Comm_size` and `MPI_Comm_rank`. We store these values in the `domain` object, which is defined in the file `data.h`.

Listing 1: Initialization of MPI

```
1     MPI_Init(&argc, &argv);
2
3     int size = 1, rank = 0;
4     MPI_Comm_size(MPI_COMM_WORLD, &size);
5     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
6
7     domain.rank = rank;
8     domain.size = size;
9
10    // read command line arguments
11    readcmdline(options, argc, argv);
12
13    // initialize sub-domain (data.{h,cpp})
14    domain.init(rank, size, options);
15    int nx = domain.nx;
16    int ny = domain.ny;
17    int N  = domain.N;
18    int nt  = options.nt;
```

In listing 2 we can see the finalization of MPI. We free the Cartesian communicator that we created in the initialization phase, and then we call `MPI_Finalize` to clean up all MPI resources.

Listing 2: Finalization of MPI

```
1     // finalize MPI
2     MPI_Comm_free(&domain.comm_cart);
3     MPI_Finalize();
```

In listing 3 we can see the welcome message that we print at the beginning of the simulation. We only print this message from the rank 0 process, to avoid cluttering the output.

Listing 3: Welcome message

```
1 if(rank == 0)
2     {
3         std::cout << std::string(80, '=') << std::endl;
4         std::cout << "                      Welcome to mini-stencil!" <<
              std::endl;
5         std::cout << "version   :: C++ MPI" << std::endl;
6         std::cout << "processes :: " << size << std::endl;
7         std::cout << "mesh      :: " << options.nx << " * " << options.nx
8                                      << " dx = " << options.dx << std::
                                         endl;
9         std::cout << "time      :: " << nt << " time steps from 0 .. "
10                                        << options.nt*options.dt << std
                                           ::endl;
11        std::cout << "iteration :: " << "CG "          << max_cg_iters
12                                     << ", Newton "    << max_newton_iters
13                                     << ", tolerance " << tolerance << std
                                         ::endl;
14        std::cout << std::string(80, '=') << std::endl;
```

```
15      }
```

1.2. Domain decomposition [10 Points]

Here we are asked to design a domain decomposition strategy that is compatible with any square grid size and any number of MPI processes while considering load balancing, communication overhead and computational efficiency. We were suggested to look at the short discussion in [**hager2010hpc**].
We can first notice that for a subdomain of size $n_x \times n_y$ we have a total computation of $n_x \cdot n_y$ while the halo exchange requires $2 \cdot (n_x + n_y)$ communicated boundary points. For a fixed local area $A = n_x n_y$, the arithmetic-geometric mean inequality gives

$$n_x + n_y \geq 2\sqrt{n_x n_y} = 2\sqrt{A},$$

with equality when $n_x = n_y$. Therefore, for a fixed local area, communication is minimized when the rectangle is as close to a square as possible. Indeed in [**hager2010hpc**] they suggest that we have to avoid slab decomposition. For example, if you split only in x, each process has a long thin strip. It computes fewer points, but the communicated boundary length stays large. A 2D Cartesian decomposition is better because both n_x and n_y decrease as the number of MPI processes increases.

Listing 4: Initialization of the sub-domain

```
1  void SubDomain::init(int mpi_rank, int mpi_size,
2                       Discretization& discretization) {
3      // Determine the number of sub-domains in the x and y dimensions
4      //       using MPI_Dims_create
5      int dims[2] = {0, 0};
6      MPI_Dims_create(mpi_size,2,dims);
7      ndomy = dims[0];
8      ndomx = dims[1];
9
10     // Create a 2D non-periodic Cartesian topology using MPI_Cart_create
11     int periods[2] = {0, 0};
12     MPI_Cart_create(MPI_COMM_WORLD,2,dims,periods,0,&comm_cart);
13
14     // Retrieve coordinates of the rank in the topology using
15     // MPI_Cart_coords
16     int coords[2] = {0, 0};
17     MPI_Cart_coords(comm_cart, mpi_rank , 2 , coords);
18     domy = coords[0] + 1;
19     domx = coords[1] + 1;
20
21     // Set neighbours for all directions using MPI_Cart_shift
22     MPI_Cart_shift(comm_cart, 0 , 1 , &neighbour_south, &neighbour_north);
23     MPI_Cart_shift(comm_cart, 1 , 1 , &neighbour_west, &neighbour_east);
24
25     // get Cartesian coordinates, but use 0-based versions for
           computations
26     int coord_y = coords[0];
27     int coord_x = coords[1];
28
29     // distribute grid points in x direction
30     int base_nx = discretization.nx / ndomx;
31     int rem_x   = discretization.nx % ndomx;
32
33     nx = base_nx + (coord_x < rem_x ? 1 : 0);
34
35     startx = coord_x * base_nx + std::min(coord_x, rem_x);
36     endx   = startx + nx - 1;
37
38     // distribute grid points in y direction
39     int base_ny = discretization.nx / ndomy;
```

```
    int rem_y   = discretization.nx % ndomy;

    ny = base_ny + (coord_y < rem_y ? 1 : 0);

    starty = coord_y * base_ny + std::min(coord_y, rem_y);
    endy   = starty + ny - 1;

    // get total number of grid points in this sub-domain
    N = nx*ny;

    rank = mpi_rank;
    size = mpi_size;
}
```

This code splits the global $N \times N$ grid into a 2D Cartesian process grid. The total number of points in each direction is divided by the number of processes along that axis, giving a base block size (`base_nx` and `base_ny`). The remainders (`rem_x` and `rem_y`) are assigned one extra grid point to the first ranks in each direction, so the subdomains differ in size by at most one cell. The expressions for `startx`, `endx`, `starty`, and `endy` then compute the global index range of each local block using prefix sums. This keeps the workload balanced while still covering the full domain without overlaps or gaps.

1.3. Linear algebra kernels [5 Points]

The linear algebra kernels were adapted to the MPI domain-decomposed setting. Most vector operations, such as `hpc_fill`, `hpc_axpy`, `hpc_scale`, `hpc_lcomb`, `hpc_copy`, `hpc_scaled_diff`, and `hpc_add_scaled_diff`, remain local. Each MPI process stores only its own subdomain, and these routines operate independently on the local entries.

The only kernels requiring MPI communication are the reduction operations `hpc_dot` and `hpc_norm2`. In both cases, each process first computes its local contribution. The local results are then combined across all MPI ranks using `MPI_Allreduce` with `MPI_SUM`. This is necessary because scalar products and norms are global quantities over the full distributed vector. In listing 5 we can see the implementation of the dot product. Each process computes the local dot product of its subdomain, and then we call `MPI_Allreduce` to sum up the local results into a global result. The same pattern is used for the 2-norm in listing 6, where we compute the local sum of squares and then take the square root of the global sum.

Listing 5: MPI parallel implementation of the dot product

```
double hpc_dot(Field const& x, Field const& y) {
    int N = y.length();
    double result = 0;
    // compute local result
    for (int i = 0; i < N; i++) {
        result += x[i] * y[i];
    }

    double global_result = 0.0;
    MPI_Allreduce(&result, &global_result, 1, MPI_DOUBLE, MPI_SUM,
                  data::domain.comm_cart);

    return global_result;
}
```

Listing 6: MPI parallel implementation of the 2-norm

```
double hpc_norm2(Field const& x) {
    int N = x.length();
    double result = 0;
    // compute local result
    for (int i = 0; i < N; i++) {
        result += x[i] * x[i];
    }
```

```
8
9       double global_result = 0.0;
10      MPI_Allreduce(&result, &global_result, 1, MPI_DOUBLE, MPI_SUM,
11                    data::domain.comm_cart);
12
13      return sqrt(global_result);
14  }
```

The conjugate-gradient solver `hpc_cg` did not require structural changes, since it calls the linear algebra kernels internally. Once the dot products and norms are made global, the CG coefficients are computed consistently across all ranks.

1.4. The diffusion stencil: Ghost cells exchange [10 Points]

Here we are asked to implement the ghost cell exchange between neighboring processes using non- blocking point-to-point communication with the objective to overlap computation and communication.
In listing 7 we provide a piece of the `diffusion` method where we tried to implement the ghost cell exchange.

Listing 7: Ghost cell exchange using non-blocking point-to-point communication

```
1       int countx = nx ; // number of interior points in x for rows
2       int county = ny ; // number of interior points in y for columns
3
4       // temporary send buffers for columns (non-contiguous in Field)
5       std::vector<double> sendW(county), sendE(county);
6       for (int j = 0; j < county; ++j) {
7           sendW[j] = s_new(0, j);          // left interior column
8           sendE[j] = s_new(nx-1, j);      // right interior column
9       }
10
11      MPI_Request reqs[8];
12      int reqcnt = 0;
13
14      // Post non-blocking receives into the boundary arrays (contiguous)
15      if (domain.neighbour_north != MPI_PROC_NULL) {
16          MPI_Irecv(&bndN[0], countx, MPI_DOUBLE, domain.neighbour_north, 0,
17                    domain.comm_cart, &reqs[reqcnt++]);
18      }
19      if (domain.neighbour_south != MPI_PROC_NULL) {
20          MPI_Irecv(&bndS[0], countx, MPI_DOUBLE, domain.neighbour_south, 0,
21                    domain.comm_cart, &reqs[reqcnt++]);
22      }
23      if (domain.neighbour_west != MPI_PROC_NULL) {
24          MPI_Irecv(&bndW[0], county, MPI_DOUBLE, domain.neighbour_west, 1,
25                    domain.comm_cart, &reqs[reqcnt++]);
26      }
27      if (domain.neighbour_east != MPI_PROC_NULL) {
28          MPI_Irecv(&bndE[0], county, MPI_DOUBLE, domain.neighbour_east, 1,
29                    domain.comm_cart, &reqs[reqcnt++]);
30      }
31
32      // Post non-blocking sends
33      if (domain.neighbour_north != MPI_PROC_NULL) {
34          MPI_Isend(&s_new(0, ny-1), countx, MPI_DOUBLE, domain.
                neighbour_north, 0,
35                    domain.comm_cart, &reqs[reqcnt++]);
36      }
37      if (domain.neighbour_south != MPI_PROC_NULL) {
38          MPI_Isend(&s_new(0, 0), countx, MPI_DOUBLE, domain.neighbour_south
                , 0,
39                    domain.comm_cart, &reqs[reqcnt++]);
40      }
```

```
41      if (domain.neighbour_west != MPI_PROC_NULL) {
42          MPI_Isend(sendW.data(), county, MPI_DOUBLE, domain.neighbour_west,
                1,
43                    domain.comm_cart, &reqs[reqcnt++]);
44      }
45      if (domain.neighbour_east != MPI_PROC_NULL) {
46          MPI_Isend(sendE.data(), county, MPI_DOUBLE, domain.neighbour_east,
                1,
47                    domain.comm_cart, &reqs[reqcnt++]);
48      }
49
50      // Wait for all posted requests
51      if (reqcnt > 0) MPI_Waitall(reqcnt, reqs, MPI_STATUSES_IGNORE);
```

This implementation exchanges the halo cells by separating the communication into two directions. The north and south boundaries can be sent directly, because the corresponding rows are contiguous in memory, while the west and east boundaries require temporary buffers since the columns of the local field are not stored contiguously. For this reason, first we pack the left and right interior columns into `sendW` and `sendE`, and then posts non-blocking receives and sends for all available neighbors. Using `MPI_Irecv` and `MPI_Isend` allows the communication to progress while the interior part of the stencil can still be computed, which is the main mechanism for overlapping communication and computation. The tags `0` and `1` distinguish row and column messages, and the checks against `MPI_PROC_NULL` ensure that boundary processes simply skip the missing neighbors. Finally, `MPI_Waitall` guarantees that all halo exchanges are complete before the updated ghost layers are used in the next stencil step.

1.5. Implement parallel I/O [10 Points]

By looking at the file `mpio_io.c` we were asked to implement the output of the computed solution with MPI-I/O.
This was done by completing the function `write_binary` in `main.cpp` as shown in listing 8.

Listing 8: MPI-IO implementation of the output

```
1  void write_binary(std::string fname, Field &u, SubDomain &domain,
2                    Discretization &options) {
3      MPI_File file;
4      MPI_File_open(MPI_COMM_WORLD, fname.c_str(),
5                    MPI_MODE_CREATE | MPI_MODE_WRONLY, MPI_INFO_NULL, &file)
                         ;
6
7      // global sizes (full global grid)
8      int sizes[2] = { options.nx, options.nx };
9      // local subarray sizes
10     int subsizes[2] = { domain.ny, domain.nx };
11     // start indices in the global array
12     int starts[2] = { domain.starty, domain.startx };
13
14     MPI_Datatype filetype;
15     MPI_Type_create_subarray(2, sizes, subsizes, starts, MPI_ORDER_C,
16                              MPI_DOUBLE, &filetype);
17     MPI_Type_commit(&filetype);
18
19     MPI_Offset disp = 0;
20     MPI_File_set_view(file, disp, MPI_DOUBLE, filetype, "native",
           MPI_INFO_NULL);
21
22     // write the local data collectively
23     MPI_File_write_all(file, u.data(), domain.N, MPI_DOUBLE,
           MPI_STATUS_IGNORE);
24
25     MPI_Type_free(&filetype);
26     MPI_File_close(&file);
```

```
27 }
```

The implementation uses MPI-IO to write the distributed field data into a single binary file. Each process defines a subarray datatype that describes its local portion of the global grid, and then collectively writes its data using `MPI_File_write_all`. The use of `MPI_Type_create_subarray` allows each process to specify its unique offset and size within the global array, enabling a seamless parallel output without the need for gathering data on a single rank.

1.6. Strong scaling [10 Points]

1.7. Weak scaling [10 Points]

1.8. Bonus [20 Points]: Overlapping computation/computation details

2. Python for High-Performance Computing [in total 40 points]

2.1. Sum of ranks: MPI collectives [5 Points]

2.2. Ghost cell exchange between neighboring processes [5 Points]

2.3. A self-scheduling example: Parallel Mandelbrot [30 Points]